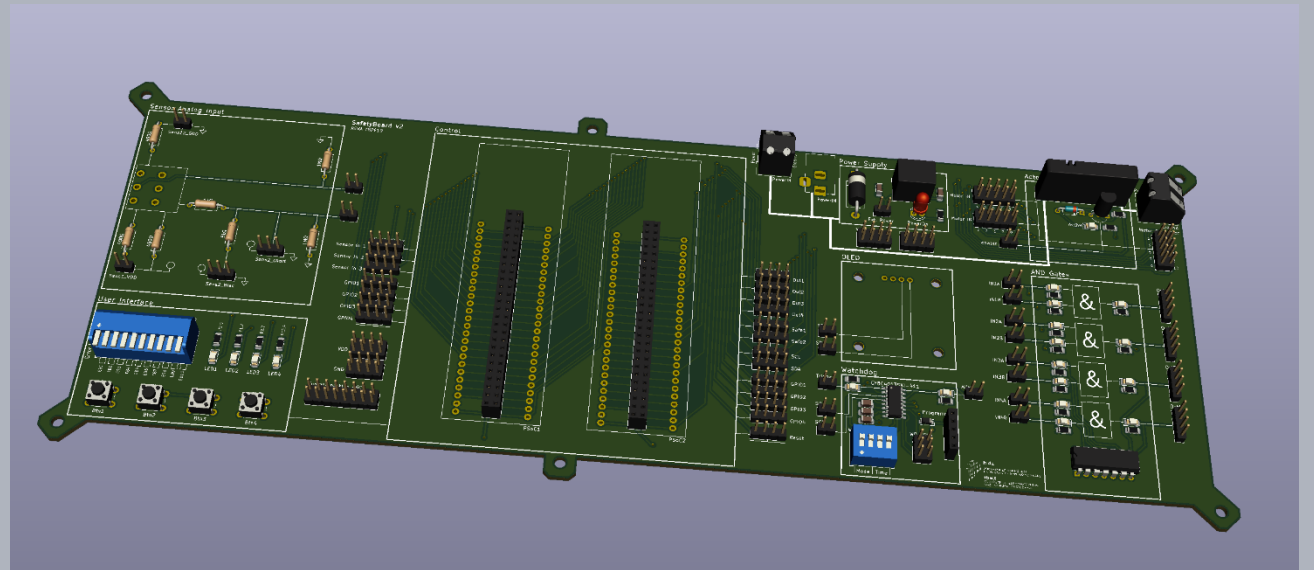


Safety in Embedded Control Systems

Moving Towards a Dual Controller Solution

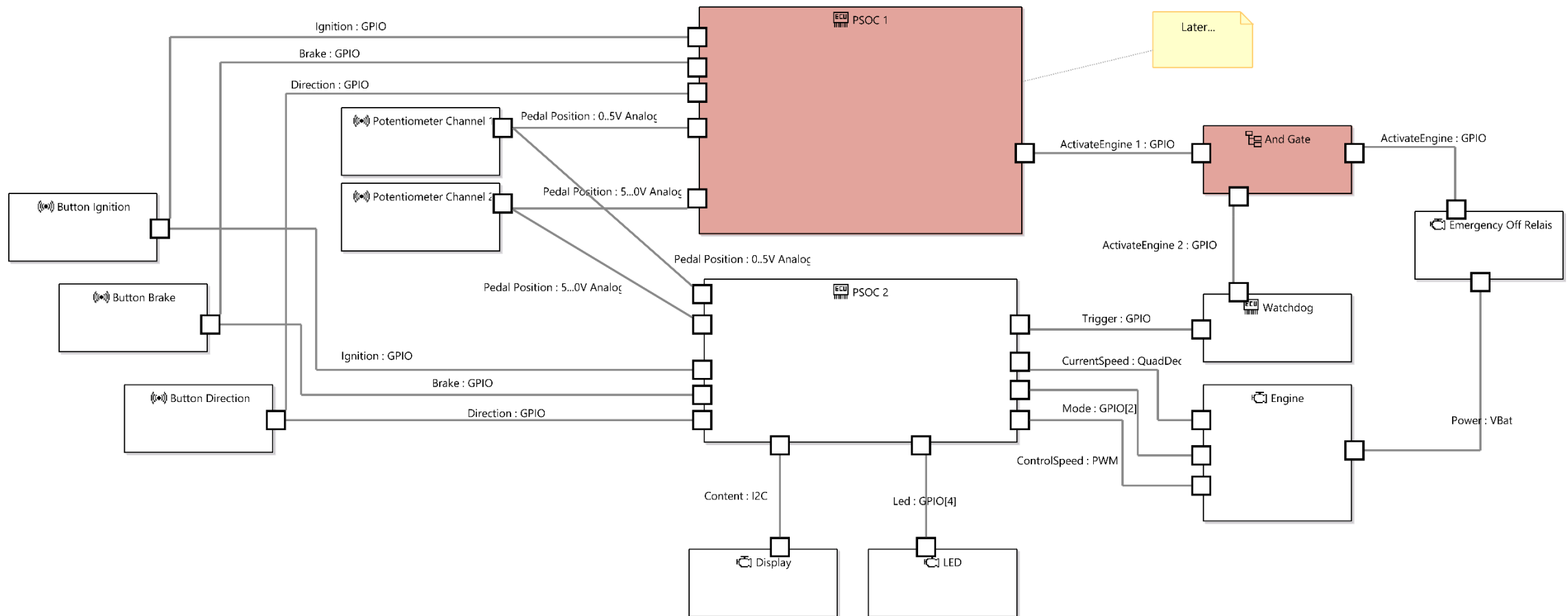
Prof. Dr.-Ing. Peter Fromm



Content

- Where we are
- Monitoring Concept
- Implementation of the Logic Monitor
- Protocol Definition
- Extensions Emergency Off
- Validation of the concept

Where we are Hardware Design



Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	- SW: Check brake pedal operation power on - SW: Ensure reliable reading of "pressed state" - HW: Provide 1oo2 brake signal - HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	   
SR2	The gas pedal signal must be reliable	C	- HW: Add resistors to detect broken / short circuit connections of gas pedal - HW: Provide 1oo2 pedal signal - HW: Use different technologies to read both channels - HW: Use inverted signals - SW: Drift of potentiometer must be detected - SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	     
SR3	The car may only start after an explicit ignition sequence	QM	- SW: A clear press of the ignition button must be detected - SW: The engine may only start if the gas pedal is depressed	 
SR4	The control speed value must be correctly calculated	C	- SW: Brake signal must override gas pedal - SW: Wrong calculations of control speed value must be detected - SW: Supervise / Limit PID Control Parameters - SW: Driving direction may only be changed if engine is stopped - SW: Reliable monitor communication link - SW: Controlspeed will be calculated with different algo's on both controllers - SW: State Machine protection	      

Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	• HW: Engine power must be deactivated in case of critical failure • HW: Engine power must be controlled by independent hardware • HW: Engine current must be supervised and limited • SW: Engine Blocking must be detected • SW: Current speed must be compared with target speed • SW: Limit reverse speed • SW: Engine must be explicitly activated	      
SR6	The system status must be shown to the driver	A	• SW: Provide status / error information to driver • SW: Detect freeze of OLED display	 
SR7	The system integrity must be supervised	C	• HW: Provide an external watchdog, which turns of the actuator in case of a critical error • HW: Power supply must be supervised, bursts must be avoided • HW: The integrity of the controller (MCU) must be supervised • SW: The integrity of the OS must be supervised • SW: ISR bursts must be detected • SW: Low Power must be detected	     

Where we are: Qualitative System Verification

System Level Stability / Watchdogs

- Frozen functions (while 1 loops) are detected
- Crash of the system (OS) is detected

PROBLEM



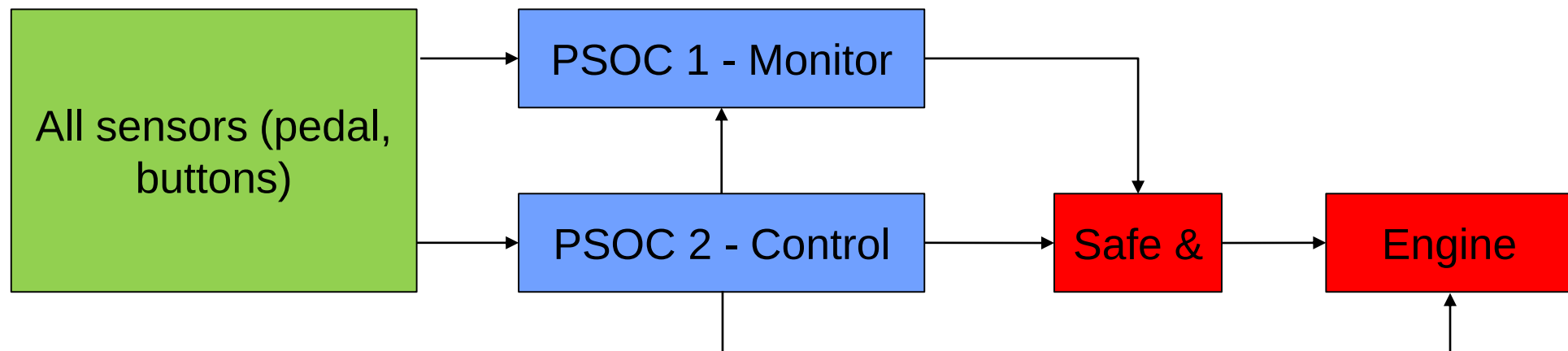
Timing problems / delays

Logic problems

Concept for PSOC1

The main task of PSOC 1 is to monitor the logic of PSOC 2 – something, which cannot be done by a simple Watchdog Controller.

- We will connect the same sensors to PSOC 1
- And implement the same software functionality on PSOC 1 (but different code, strictly spoken)
- PSOC 2 will transfer all relevant internal states to PSOC 1
- PSOC 1 will calculate the same states and compare the result with the transmitted states
- If they are equal, the system is in a good overall state. If not, PSOC 1 assumes a problem with the main PSOC 2 and triggers a transition into the safe state



Other Actors

Problem: We have a couple of other actors in the game

- OLED Display
- LED's

Luckily we have
the RTE –
simply set the
OutPorts to 0

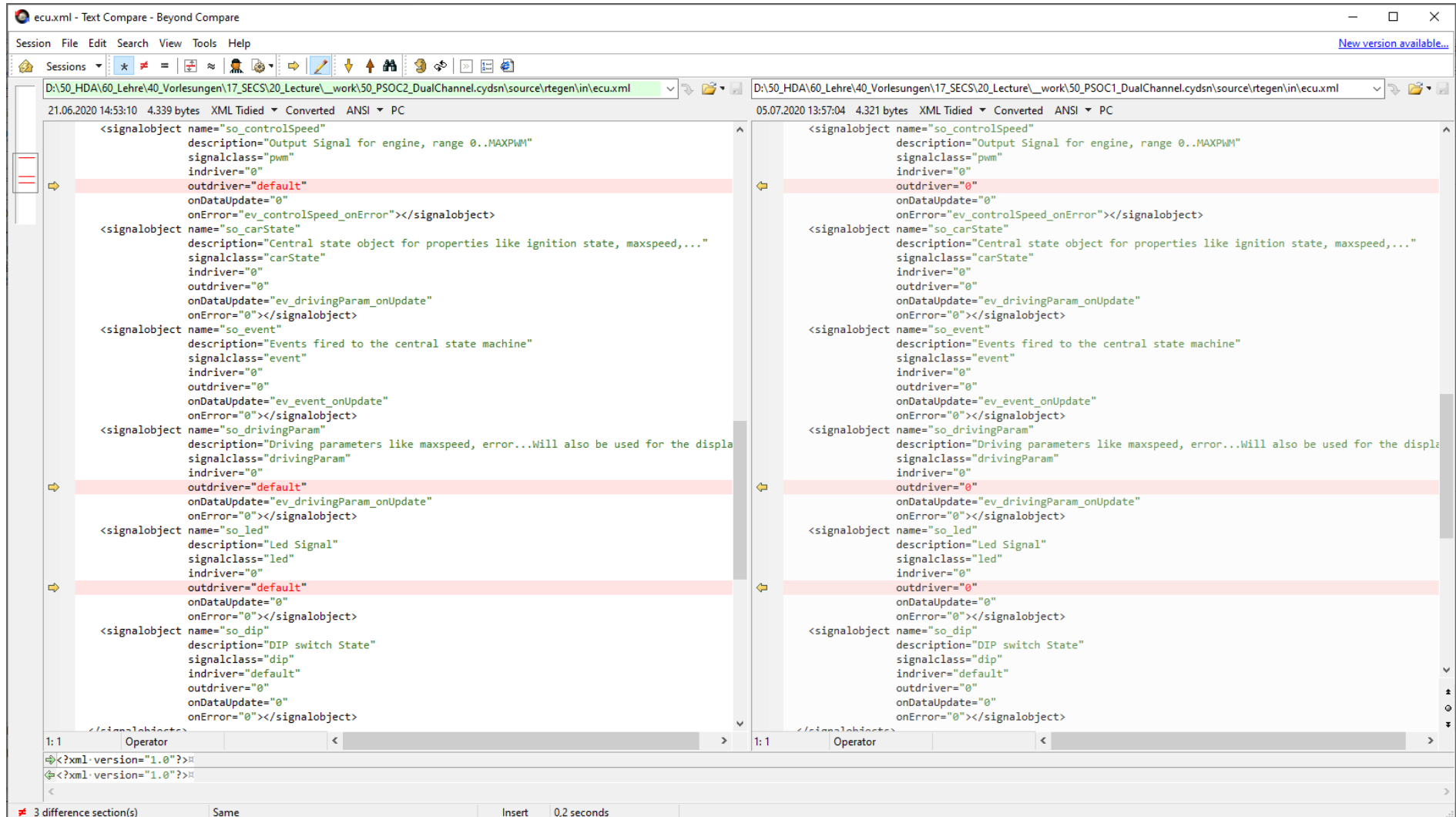
Only the LED's
are a real issue
– simply set
them to inputs



Driving actors
from 2 MCU's –
not a good idea!

Sh...we still
have some
hardware
interference...

And off go the actors...



Monitoring Concept

What data needs to be compared between PSOC 2 and PSOC1

- In a first attempt, we will focus on the CARSTATE and TARGETSPEED value, as they are internally calculated



Communication interface design

- As we only want to transmit from PSOC Control (2) to PSOC Monitor (1), we will add the following signals
 - so_uart_tx on PSOC 2
 - so_uart_rx on PSOC1
- They will both be based on the same signal class UARTPROT and share the same protocol parser codebase

Uartprot signal type and protocol stream

```
1 #define PROT_SIZERXBUF    100                /**< \brief Size of the UART reception buffer */
   #define UARTPROT_SOP      'S'
   #define UARTPROT_EOP1     0x0D
   #define UARTPROT_EOP2     0x0E

1 /**
   * This data structure will contain all relevant data to be exchanged between the PSOC 2 (Control) and PSOC (1) Monitor controller
   */
   typedef struct
1 {
       CAR_state_t    m_state;
       sint16_t        m_targetspeed;
- } UARTPROT_data_t;

   #define UARTPROT_INIT_DATA ((UARTPROT_data_t){0})
```

SOP	m_state	LO(m_targetspeed)	HI(m_targetspeed)	CRC	EOP1	EOP2
-----	---------	-------------------	-------------------	-----	------	------

Parser

```
/**
 * Default IN driver API - may be deleted if not required
 */
inline RC_t UARTPROT_driverOut(const UARTPROT_data_t *const data)
{

    //Scale application data to drive format
    //Write scaled data to driver

    uint8_t byte = 0;

    UART_PSOC_WriteTxData(UARTPROT_SOP);

    byte = (uint8_t) data->m_state;
    UART_PSOC_WriteTxData(byte);

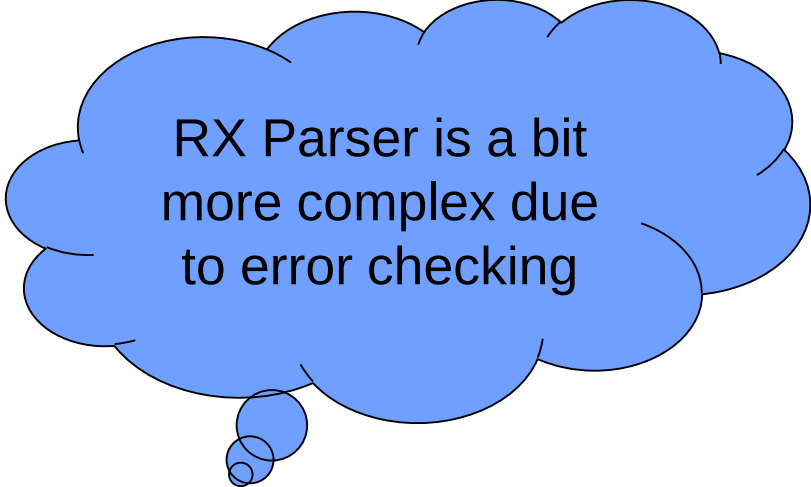
    byte=(uint8_t) (data->m_targetspeed);
    UART_PSOC_WriteTxData(byte);

    byte=(uint8_t) (data->m_targetspeed >> 8);
    UART_PSOC_WriteTxData(byte);

    byte=calcCRC(data);
    UART_PSOC_WriteTxData(byte);

    UART_PSOC_WriteTxData(UARTPROT_EOP1);
    UART_PSOC_WriteTxData(UARTPROT_EOP2);

    return RC_SUCCESS;
}
```



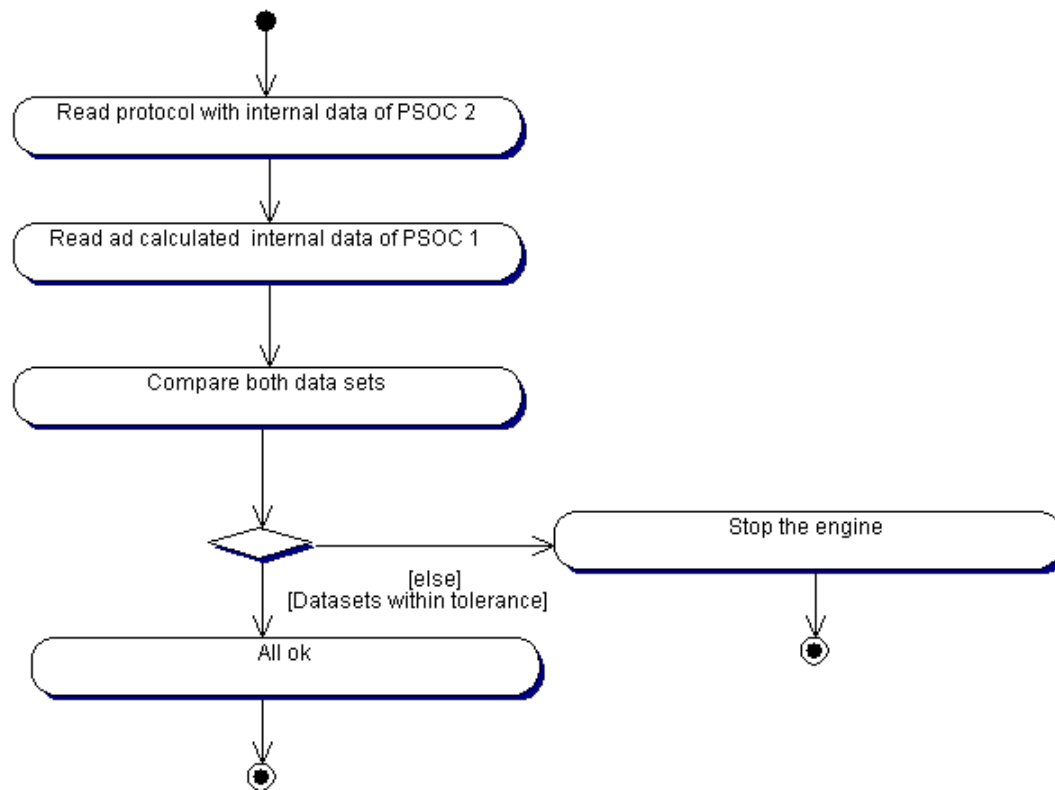
RX Parser is a bit more complex due to error checking

CRC calculation

As data may be corrupted during transmission, we protect the payload using a simple XOR CRC algorithm

```
uint8_t calcCRC(const UARTPROT_data_t *const data)
{
    uint8_t crc = 0;
    crc = (uint8_t)data->m_state ^ (uint8_t) data->m_targetspeed ^ (uint8_t) (data->m_targetspeed >> 8);
    return crc;
}
```

Processing the data



```
#define SPEED_MAXDELTA 20

void COM_receive_run()
{
    //Read data out of buffer
    RTE_UARTPROT_pullPort(&SO_UARTPROT_RX_signal);
    UARTPROT_data_t prot = RTE_UARTPROT_get(&SO_UARTPROT_RX_signal);

    LOG_I("COM_receive_run", "S: %d TS: %d", prot.m_state, prot.m_targetspeed);

    //Get comparison data
    CARSTATE_data_t state = RTE_CARSTATE_get(&SO_CARSTATE_signal);
    SPEED_data_t targetspeed = RTE_SPEED_get(&SO_TARGETSPEED_signal);

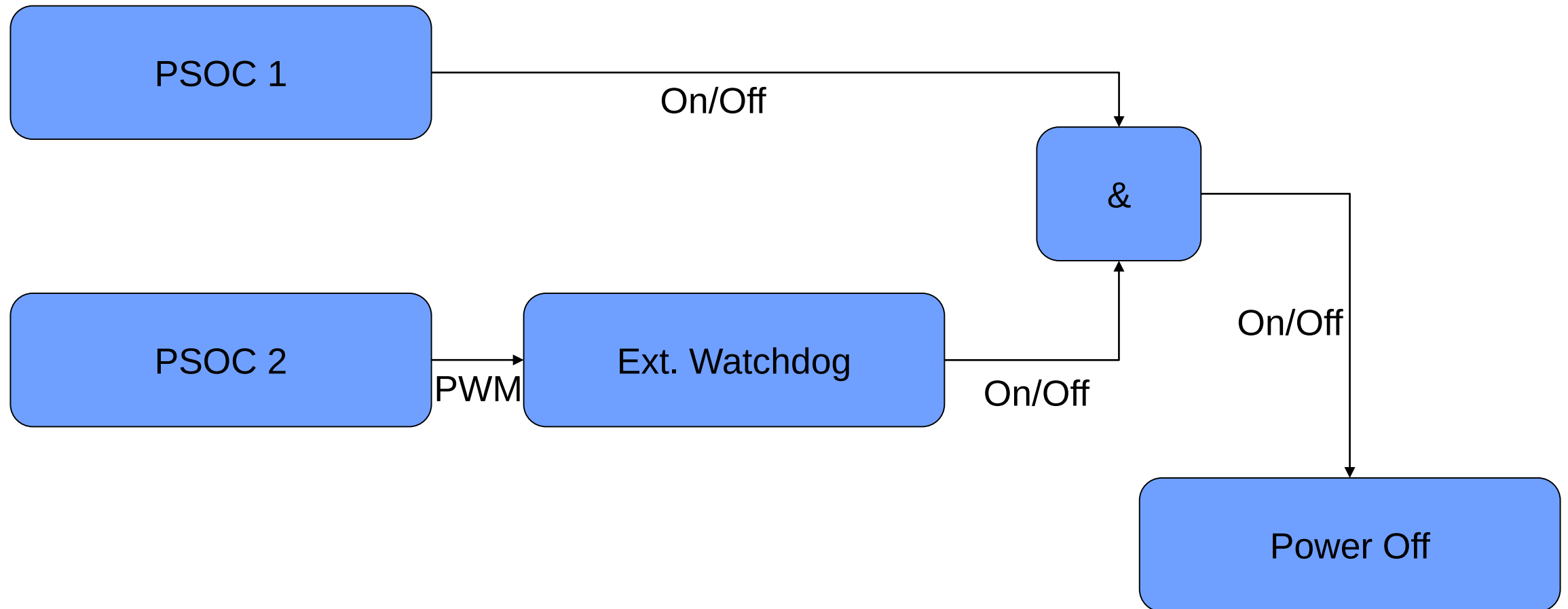
    if (prot.m_state != state.m_state)
    {
        LOG_E("COM_receive_run", "State different - rx: %d, int: %d", prot.m_state, state.m_state);
        ERROR_centralHandler(ERROR_LOGIC);
    }

    if (prot.m_targetspeed != targetspeed.m_rpm)
    {
        LOG_E("COM_receive_run", "Speed different - rx: %d, int: %d", prot.m_targetspeed, targetspeed.m_rpm);
    }

    sint16_t speedDelta = abs(prot.m_targetspeed - targetspeed.m_rpm);

    if (speedDelta > SPEED_MAXDELTA)
    {
        ERROR_centralHandler(ERROR_LOGIC);
    }
}
```

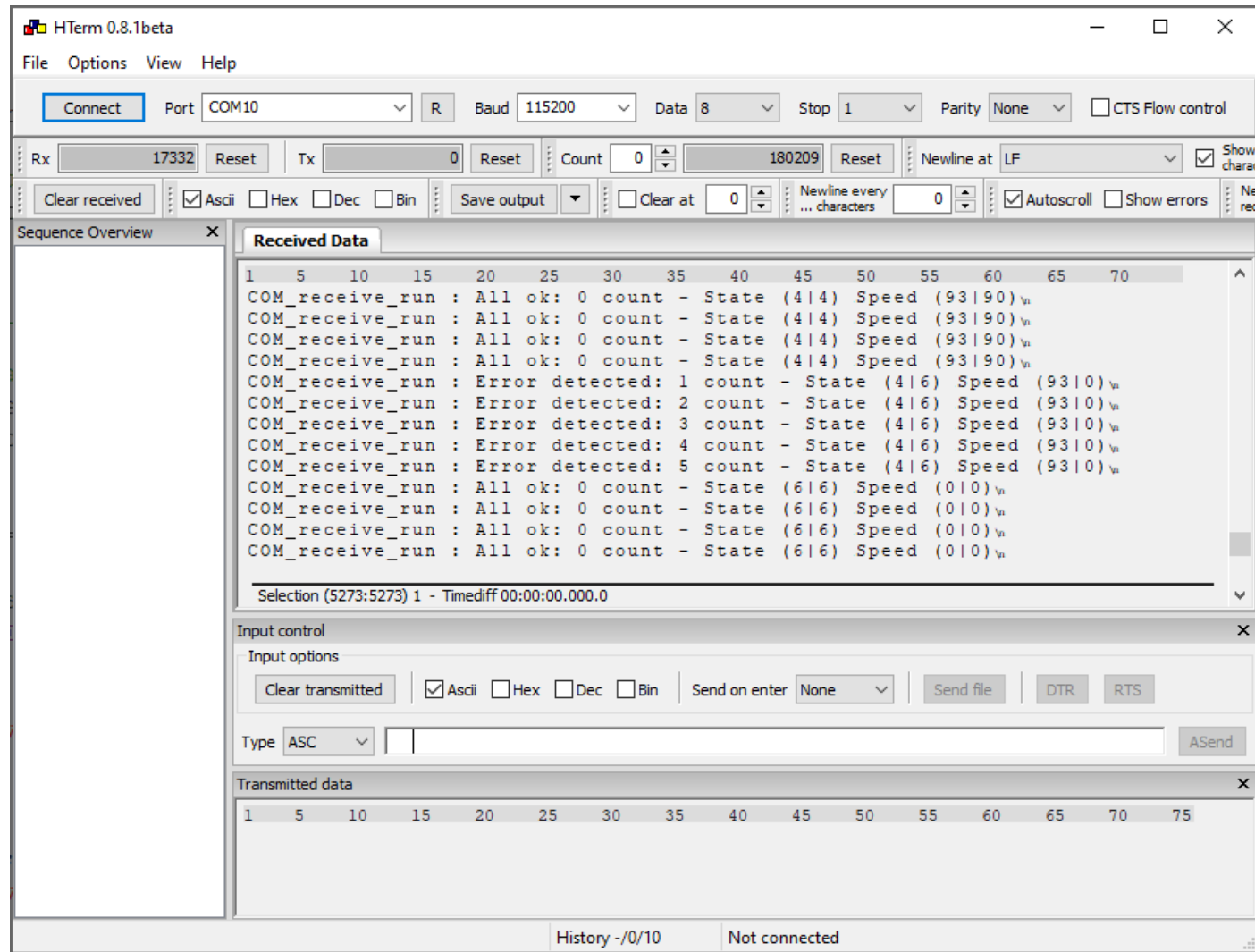
Emergency Off



Let's perform a first smoke test

Testcase	Expected Output	Result
Wrong input Signal	System turns off	Partly Ok But monitor turns off power?
Blocking Engine	System turns off	Partly Ok But wrong behavior in second cycle – power off.
PSOC 2 freezes	System turns off	Ok
PSOC 1 freezes	System turns off	NOK, as PSOC 1 has a too simple safety pin
PSOC 2 wrong timebase	System turns off	NOK – as the targetvalue is correct (only currentspeed and measuredspeed are effected by the wrong time base)

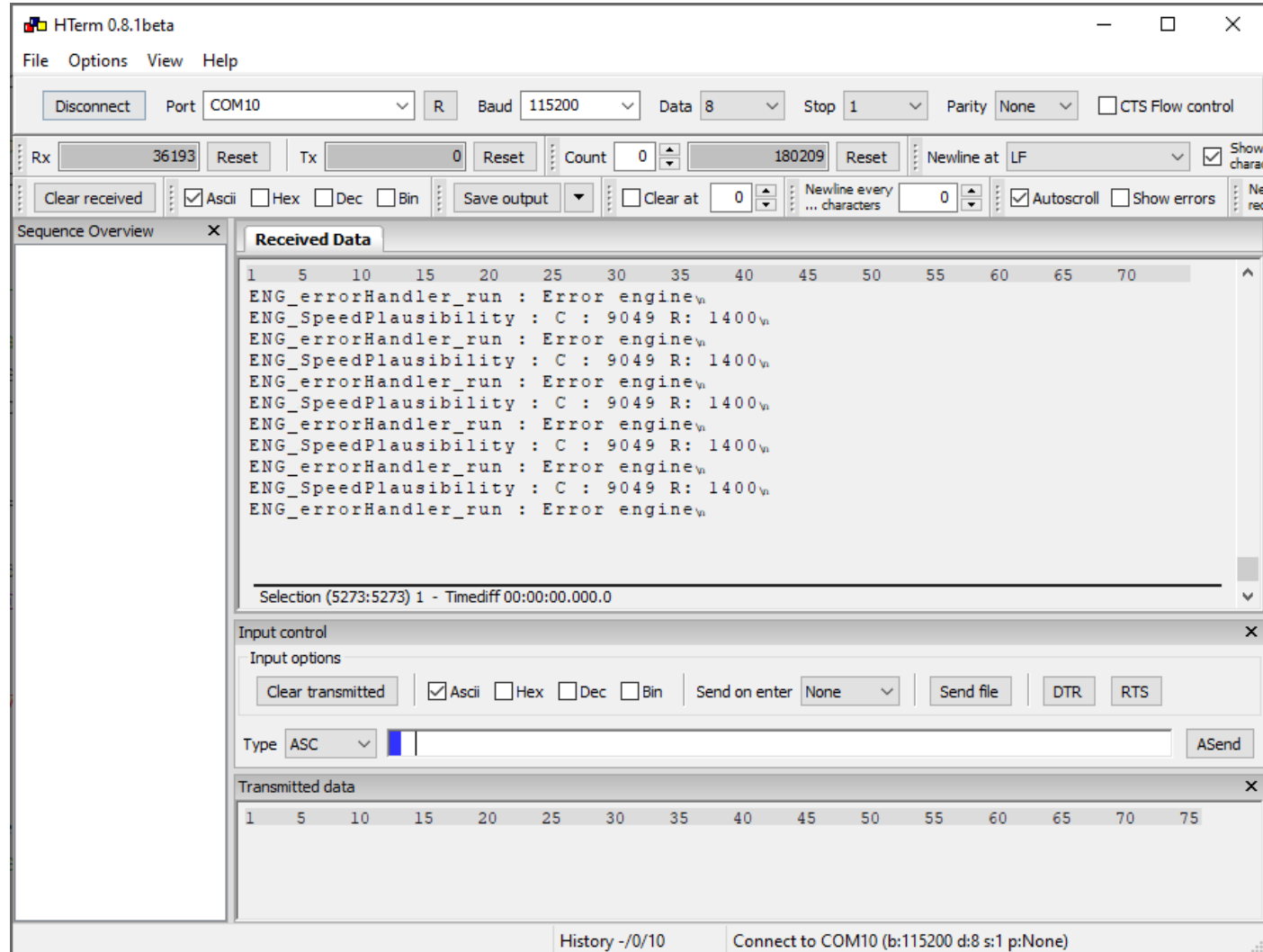
Analysis of wrong input signal



Monitor detects wrong input faster than transmission of control PSOC

ADC are scanned every 10ms, data is transmitted every 100ms

Analysis of second cycle after blocking engine



The screenshot shows the HTerm 0.8.1beta terminal window. The top menu bar includes File, Options, View, and Help. Below the menu is a toolbar with buttons for Disconnect, Port (COM10), Baud (115200), Data (8), Stop (1), Parity (None), and CTS Flow control. The main window is divided into two panes: Sequence Overview on the left and Received Data on the right. The Received Data pane shows a log of engine error messages:

```
1 5 10 15 20 25 30 35 40 45 50 55 60 65 70
ENG_errorHandler_run : Error engine_v
ENG_SpeedPlausibility : C : 9049 R: 1400_v
ENG_errorHandler_run : Error engine_v
ENG_SpeedPlausibility : C : 9049 R: 1400_v
ENG_errorHandler_run : Error engine_v
ENG_SpeedPlausibility : C : 9049 R: 1400_v
ENG_errorHandler_run : Error engine_v
ENG_SpeedPlausibility : C : 9049 R: 1400_v
ENG_errorHandler_run : Error engine_v
ENG_SpeedPlausibility : C : 9049 R: 1400_v
ENG_errorHandler_run : Error engine_v
```

Below the log, there is a selection bar with the text "Selection (5273:5273) 1 - Timediff 00:00:00.000.0". At the bottom of the window, there is a status bar showing "History -/0/10" and "Connect to COM10 (b:115200 d:8 s:1 p:None)".

Reference Engine
Controller on
Monitor needs to be
reset

Problem corrupted time base

Possible solution

- Let's also transmit the measured speed
- As this a very dangerous error, the engine should stop as fast as possible
- Unfortunately fast does not mean immediately!

ITerm 0.8.1beta

File Options View Help

Connect Port COM10 R Baud 115200 Data 8 Stop 1 Parity None ☐ CTS Flow control

Rx 2798 Reset Tx 0 Reset Count 0 180209 Reset Newline at LF ☒ Show charac

☐ Clear received ☒ Ascii ☐ Hex ☐ Dec ☐ Bin Save output ☐ Clear at 0 Newline every ... characters 0 ☒ Autoscroll ☐ Show errors Ne rec

Sequence Overview

Received Data

```
1 5 10 15 20 25 30 35 40 45 50 55 60 65 70
ENG_SpeedPlausibility : C : 8190 R: 2800
ENG_errorHandler_run : Error engine
COM_receive_run : Error detected: 10 count - State (4|4) Speed (100|100) C
urrent (-1835|-197)
ERROR_centralHandler : Stopped engine
ENG_SpeedPlausibility : C : 10000 R: 4150
ENG_errorHandler_run : Error engine
ENG_SpeedPlausibility : C : 10000 R: 2800
ENG_errorHandler_run : Error engine
ENG_SpeedPlausibility : C : 10000 R: 2800
ENG_errorHandler_run : Error engine
ENG_SpeedPlausibility : C : 10000 R: 1400
ENG_errorHandler_run : Error engine
```

Selection (5273:5273) 1 - Timediff 00:00:00.000.0

Input control

Input options

☒ Ascii ☐ Hex ☐ Dec ☐ Bin Send on enter None

Type ASC

Transmitted data

1 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75

History -0/10 Not connected

Remaining Problem 1

What happens if no data is being transmitted?

I.e. the communication link between the two controller breaks?

- Well, we could check the age of the transmitted signal (better)
- Or rely on the detection of a different state as soon the car starts the ignition

Remaining Problem 2

Reset of Monitor

The monitor is implemented as a single turn of device.

In case of an error, it turns of the engine and keeps this state until a reset occurs.

Idea

- Restart monitor in case PSOC 2 gets a reset or ignition signal.

Now we reach the typical safety challenge: Availability of the system versus safety.
Often not that easy to find a solution.

Final System: Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	- SW: Check brake pedal operation power on - SW: Ensure reliable reading of "pressed state" - HW: Provide 1oo2 brake signal - HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	   
SR2	The gas pedal signal must be reliable	C	- HW: Add resistors to detect broken / short circuit connections of gas pedal - HW: Provide 1oo2 pedal signal - HW: Use different technologies to read both channels - HW: Use inverted signals - SW: Drift of potentiometer must be detected - SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	     
SR3	The car may only start after an explicit ignition sequence	QM	- SW: A clear press of the ignition button must be detected - SW: The engine may only start if the gas pedal is depressed	 
SR4	The control speed value must be correctly calculated	C	- SW: Brake signal must override gas pedal - SW: Wrong calculations of control speed value must be detected - SW: Supervise / Limit PID Control Parameters - SW: Driving direction may only be changed if engine is stopped - SW: Reliable monitor communication link - SW: Controlspeed will be calculated with different algo's on both controllers - SW: State Machine protection	      

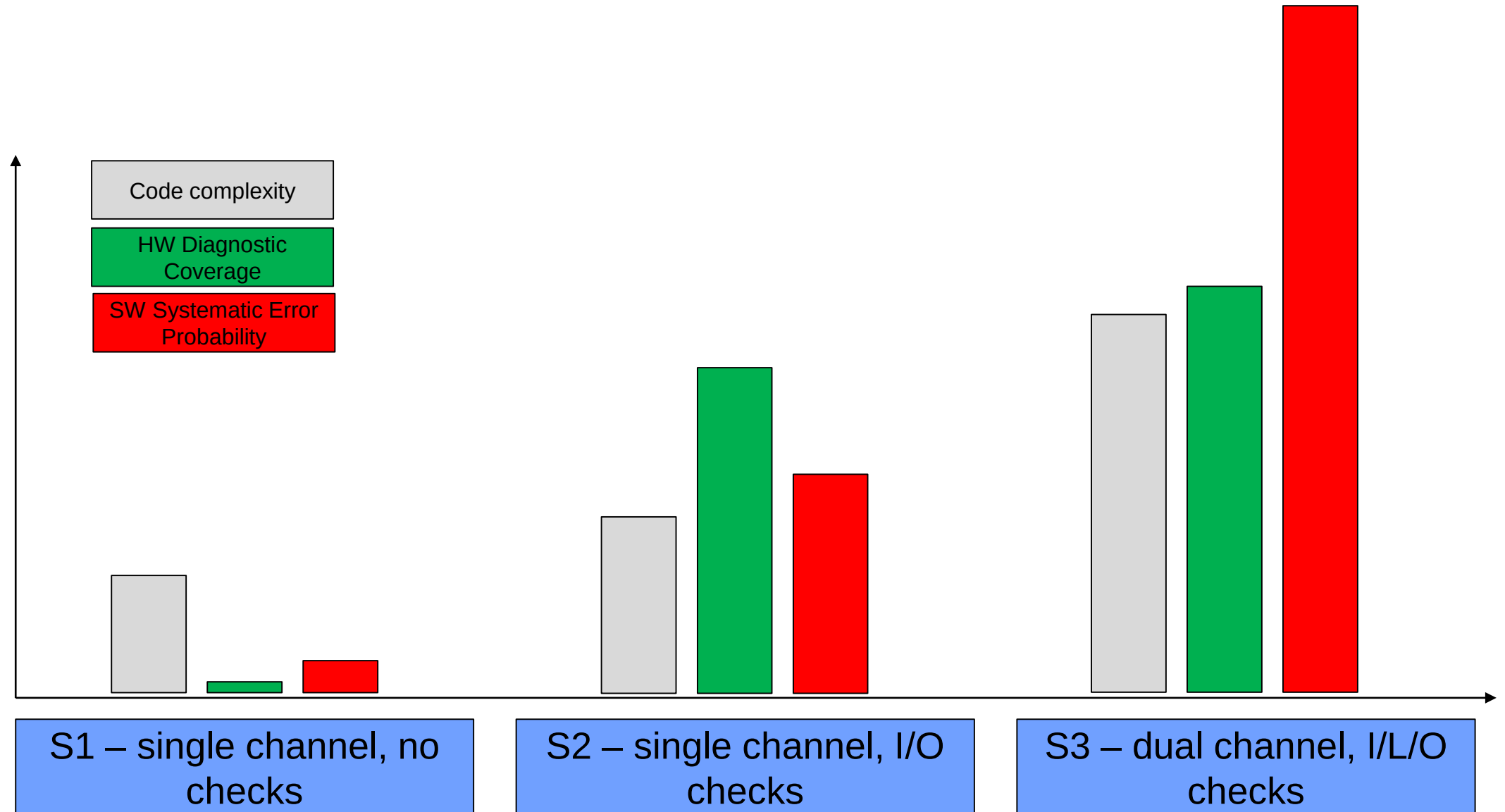
Final System: Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	- HW: Engine power must be deactivated in case of critical failure - HW: Engine power must be controlled by independent hardware - HW: Engine current must be supervised and limited - SW: Engine Blocking must be detected - SW: Current speed must be compared with target speed - SW: Limit reverse speed - SW: Engine must be explicitly activated	      
SR6	The system status must be shown to the driver	A	- SW: Provide status / error information to driver - SW: Detect freeze of OLED display	 
SR7	The system integrity must be supervised	C	- HW: Provide an external watchdog, which turns of the actuator in case of a critical error - HW: Power supply must be supervised, bursts must be avoided - HW: The integrity of the controller (MCU) must be supervised - SW: The integrity of the OS must be supervised - SW: ISR bursts must be detected - SW: Low Power must be detected	     

Reliability / Complexity / Availability



Which one to chose?



Effort and Risk Evaluation

- The efforts for implementing the second controller are pretty high
- Many “false positive” signals stopped the engine although not required
- State machine variable → better solution MPU
- Corrupted time base → Better solution Safety RTOS

As an engineer, we have to chose our way!



Wrap up – What did we do?

Mitigation of sensor faults

- Detection of short circuits and broken connections using resistors
- Detection of wear-off using redundant potentiometers
- Self-test of brake pedal

Mitigation of user faults

- Long press of ignition button
- Change of direction only in case of stand still

Mitigation of actor faults

- Robustness clippers in engine controller
- Comparison of PID result with open loop calibration curve

Mitigation of logic and system faults

- Alive Watchdog (external / internal)
- Second controller act as monitor

Remember: Main Requirements for a Safety Architecture

Avoid Dangerous Failures

Reliable detection and handling of stochastic faults and errors (focus single faults)
E.g. hardware failures, communication errors,...



Reduction / avoidance of systematic errors.
E.g. programming bugs

TODO!

In case of a critical failure: Transition into a safe state.

